DS 8001 — Lecture 7: Recommender Systems

Exercises with Worked Solutions

Conventions used throughout. Users are indexed by x, y and items (movies) by i, j, k, m; r_{xi} is the rating of user x for item i and $\hat{r}_{xi}$ is a prediction of it, as in the lecture. Numbers are rounded to the digits shown, but every computation is carried out with unrounded values; all of them were rechecked by a script.

Exercise 1: Content-Based RecSys

Ex1: Content-Based RecSys

Imagine you have a dataset containing information about various movies. Each movie has features such as genre, director, main actors and a brief description, and each user has a list of the movies they have rated.

We want a content-based recommender system that suggests new movies to a user based on the movies the user rated highly. A user has just joined and rated three movies: "Inception" (5 stars), "The Dark Knight" (4 stars) and "Interstellar" (5 stars). The five candidate movies below are unseen by this user. Use only the features in the table (the genre tags as listed and one main actor per movie):

Movie	Genre tags	Director	Main actor
Inception	Sci-Fi, Thriller	Christopher Nolan	Leonardo DiCaprio
The Dark Knight	Action, Crime	Christopher Nolan	Christian Bale
Interstellar	Sci-Fi, Drama	Christopher Nolan	Matthew McConaughey
Tenet	Sci-Fi, Action, Thriller	Christopher Nolan	John David Washington
The Prestige	Drama, Thriller	Christopher Nolan	Christian Bale
The Matrix	Sci-Fi, Action	Lana and Lilly Wachowski	Keanu Reeves
Gravity	Sci-Fi, Drama, Thriller	Alfonso Cuarón	Sandra Bullock
Titanic	Drama, Romance	James Cameron	Leonardo DiCaprio

Tasks:

(a) Define an item profile for each of the eight movies based on genre, director and main actor: a binary vector with one coordinate per feature value (each genre tag, each director – the Wachowskis count as one – and each actor), equal to 1 if the movie has that feature and 0 otherwise. How many coordinates does a profile have?

(b) Construct the user profile x as the weighted average of the item profiles i_j of the three rated movies, weighting each movie j by its rating r_j: $x = \sum_j r_j \, i_j \,/\, \sum_j r_j$.

(c) Using cosine similarity, $\cos(x, i) = x \cdot i \,/\, (\|x\| \, \|i\|)$, score the five candidate movies and list the top 3 movie recommendations for this user.

(d) Which feature has the largest weight in x, and why? Which weakness of content-based recommendation does your top 3 illustrate? Could this method recommend anything to a user who has not rated any movie? How could a hybrid system help?

Solution

Idea. A content-based recommender never looks at other users. It describes every movie by a vector of features (the *item profile*), summarizes the user by one vector in the *same* feature space (the *user profile*), and recommends the unseen movies whose profiles point in the most similar direction.

(a) Listing the feature values. We collect every distinct value that occurs in the table. A value shared by several movies (Nolan directs five of them, DiCaprio and Bale each appear twice) gets *one* coordinate:

- genre tags (6): Sci-Fi, Thriller, Action, Crime, Drama, Romance;
- directors (4): Nolan, Wachowski, Cuarón, Cameron;
- main actors (6): DiCaprio, Bale, McConaughey, Washington, Reeves, Bullock.

So every item profile has $6+4+6 = \mathbf{16}$ coordinates. Each profile is a row of the table below (1 = the movie has the feature; grey 0 = it does not). The last row anticipates part (b).

	genre tags						director				main actor					
Movie	SF	Thr	Act	Cri	Dra	Rom	Nol	Wac	Cua	Cam	DiC	Bal	McC	Was	Ree	Bul
Inception	1	1	0	0	0	0	1	0	0	0	1	0	0	0	0	0
The Dark Knight	0	0	1	1	0	0	1	0	0	0	0	1	0	0	0	0
Interstellar	1	0	0	0	1	0	1	0	0	0	0	0	1	0	0	0
Tenet	1	1	1	0	0	0	1	0	0	0	0	0	0	1	0	0
The Prestige	0	1	0	0	1	0	1	0	0	0	0	1	0	0	0	0
The Matrix	1	0	1	0	0	0	0	1	0	0	0	0	0	0	1	0
Gravity	1	1	0	0	1	0	0	0	1	0	0	0	0	0	0	1
Titanic	0	0	0	0	1	1	0	0	0	1	1	0	0	0	0	0
$14\,x$ (part b)	10	5	4	4	5	0	14	0	0	0	5	4	5	0	0	0

A binary vector with n ones has length $\sqrt{n}$: Tenet and Gravity have 5 ones, so $\|i\| = \sqrt{5} \approx 2.236$; every other profile has 4 ones, so $\|i\| = 2$.

(b) The ratings are $5, 4, 5$, so the weights are $5/14$, $4/14$ and $5/14$ (they sum to 1, which is what makes this an *average*):

$$x = \tfrac{5}{14}\, i_{\text{Inception}} + \tfrac{4}{14}\, i_{\text{Dark Knight}} + \tfrac{5}{14}\, i_{\text{Interstellar}}.$$

Coordinate by coordinate, a feature receives the total rating of the rated movies that have it, divided by 14:

Feature	rated movies having it	weight in x
Nolan (director)	all three	$(5+4+5)/14 = 1$
Sci-Fi	Inception, Interstellar	$(5+5)/14 = 10/14 \approx 0.714$
Thriller, DiCaprio	Inception	$5/14 \approx 0.357$
Drama, McConaughey	Interstellar	$5/14 \approx 0.357$
Action, Crime, Bale	The Dark Knight	$4/14 \approx 0.286$
the other 7 features	none	0

Answer. $x = \frac{1}{14}\,(10, 5, 4, 4, 5, 0 \mid 14, 0, 0, 0 \mid 5, 4, 5, 0, 0, 0)$ in the coordinate order of the table, i.e. Sci-Fi 0.714, Thriller 0.357, Action 0.286, Crime 0.286, Drama 0.357, Nolan 1, DiCaprio 0.357, Bale 0.286, McConaughey 0.357, all others 0.

(c) A simplification. Cosine similarity does not change when a vector is multiplied by a positive number, so we may use the integer vector $\tilde{x} = 14x$ instead of x:

$$\|\tilde{x}\|^2 = 10^2 + 5^2 + 4^2 + 4^2 + 5^2 + 14^2 + 5^2 + 4^2 + 5^2 = 444, \qquad \|\tilde{x}\| = \sqrt{444} \approx 21.07.$$

Because the item profiles are binary, $\tilde{x} \cdot i$ is simply the sum of the entries of $\tilde{x}$ at the features the movie has:

Candidate	features with nonzero weight in $\tilde{x}$	$\tilde{x} \cdot i$	$\|i\|$	$\cos(x,i) = \dfrac{\tilde{x} \cdot i}{21.07\,\|i\|}$
Tenet	Sci-Fi 10, Action 4, Thriller 5, Nolan 14	33	$\sqrt{5}$	$33/47.12 = \mathbf{0.700}$
The Prestige	Drama 5, Thriller 5, Nolan 14, Bale 4	28	2	$28/42.14 = \mathbf{0.664}$
Gravity	Sci-Fi 10, Drama 5, Thriller 5	20	$\sqrt{5}$	$20/47.12 = \mathbf{0.424}$
The Matrix	Sci-Fi 10, Action 4	14	2	$14/42.14 = 0.332$
Titanic	Drama 5, DiCaprio 5	10	2	$10/42.14 = 0.237$

(Washington, Reeves, Bullock, Romance and the three other directors have weight 0 in x, so they add nothing to the dot products; they only enlarge $\|i\|$.)

Answer. Top 3 recommendations: **1. Tenet** (0.700), **2. The Prestige** (0.664), **3. Gravity** (0.424); then The Matrix (0.332) and Titanic (0.237).

(d) Largest weight. The director coordinate *Nolan* has weight 1, the largest possible: all three rated movies are Nolan films, so it collects the full rating mass 14/14. Every other feature occurs in at most two of them.

Weakness illustrated: overspecialization. The two Nolan films take the first two places. The Prestige comes second although it has no Sci-Fi tag, the user's strongest genre: the Nolan coordinate alone contributes 14 of its 28 points. A content-based recommender only finds "more of the same": it never recommends items outside the user's content profile, and it cannot use the quality judgments of other users. (If the Nolan coordinate is removed, the order becomes Tenet 0.603, Gravity 0.568, The Prestige 0.513, so the shared director is what lifts The Prestige above Gravity.)

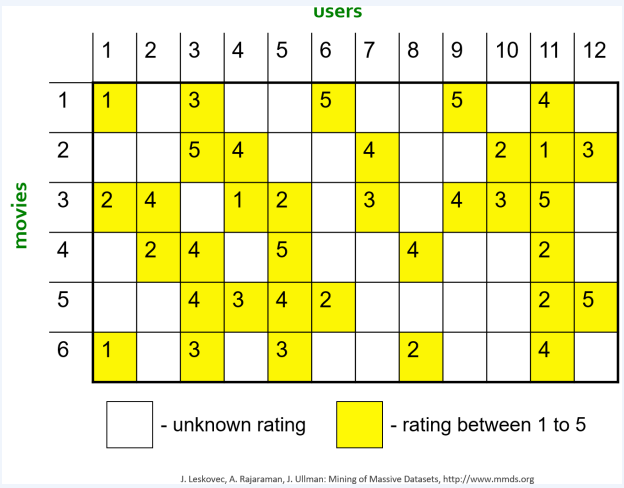
A user with no ratings. No. With no rated movies the weighted average $\sum_j r_j i_j / \sum_j r_j$ is 0/0: there is no user profile to compare with. Content-based methods survive a *new item* (its profile comes from its features) but not a *new user*.

How a hybrid helps. A hybrid combines this recommender with others, for example by a linear combination of their predictions. Collaborative filtering adds what similar users liked, which can lie outside the user's content profile and reflects quality judgments, so it counters overspecialization. For a brand-new user, demographic information or non-personalized lists ("most popular") provide recommendations until the user has rated enough movies.

Exercise 2: Collaborative Filtering

Ex2: Collaborative Filtering

Consider the user-item rating matrix below, from a movie recommendation system (it is the matrix of the lecture's item-item example). Each column represents a user, each row represents a movie, and each yellow cell contains a rating (1–5 stars); a cell is blank if the user hasn't rated the movie.



For User 1, use user-user collaborative filtering to predict their rating for Movie 4, which they haven't rated yet.

Measure the similarity of users x and y with the Pearson correlation coefficient from the lecture,

$$sim(x,y) = \frac{\sum_{s \in S_{xy}} (r_{xs} - \bar{r}_x)(r_{ys} - \bar{r}_y)}{\sqrt{\sum_{s \in S_{xy}} (r_{xs} - \bar{r}_x)^2} \sqrt{\sum_{s \in S_{xy}} (r_{ys} - \bar{r}_y)^2}},$$

where r_{xs} is the rating of user x for movie s, S_{xy} is the set of movies rated by both x and y, and $\bar{r}_x$ is the average of *all* the ratings of user x (not only those in S_{xy}).

(a) Calculate the similarity between User 1 and each user who has rated Movie 4.

(b) Identify the top 3 users most similar to User 1 (the neighborhood N, $k = |N| = 3$).

(c) Predict User 1's rating for Movie 4 using the weighted average of the ratings given to Movie 4 by these top 3 similar users, weighted by their similarities: $\hat{r}_{xi} = \sum_{y \in N} s_{xy}\, r_{yi} \,/\, \sum_{y \in N} s_{xy}$, with $s_{xy} = sim(x,y)$.

(d) How many movies does each user in N have in common with User 1? Show that if S_{xy} contains a single movie, then $sim(x,y)$ is +1 or −1 (or undefined). How much do you trust your prediction, and which drawback of collaborative filtering does this illustrate?

Solution

Reading the data. The matrix is the transpose of the lecture's utility matrix, so a *user's* ratings are a *column*. User 1 rated Movies 1, 3 and 6 (ratings 1, 2, 1), so $\bar r_1 = 4/3$ and User 1's deviations from their mean are

$$r_{1,1} - \bar r_1 = -\tfrac{1}{3}, \qquad r_{1,3} - \bar r_1 = +\tfrac{2}{3}, \qquad r_{1,6} - \bar r_1 = -\tfrac{1}{3}.$$

Row 4 shows that Movie 4 was rated by Users 2, 3, 5, 8 and 11. Their columns:

User y	ratings (movie: stars)	mean $\bar r_y$	$r_{y,4}$	S_{1y} (shared with User 1)
2	3:4, 4:2	6/2 = 3	2	$\{3\}$
3	1:3, 2:5, 4:4, 5:4, 6:3	19/5 = 3.8	4	$\{1,6\}$
5	3:2, 4:5, 5:4, 6:3	14/4 = 3.5	5	$\{3,6\}$
8	4:4, 6:2	6/2 = 3	4	$\{6\}$
11	1:4, 2:1, 3:5, 4:2, 5:2, 6:4	18/6 = 3	2	$\{1,3,6\}$

Note that the means use *all* of a user's ratings, including the rating of Movie 4, while the sums in *sim* run only over S_{1y}.

(a) For each user we list the two deviation vectors on S_{1y} and evaluate the formula (numerator / product of the two root-sums).

- **User 2**, $S = \{3\}$: deviations $(\frac{2}{3})$ and $(4-3) = (1)$. $\quad sim = \dfrac{\frac{2}{3}\cdot 1}{\sqrt{4/9}\,\sqrt{1}} = \dfrac{2/3}{2/3} = 1.$
- **User 3**, $S = \{1,6\}$: deviations $(-\frac{1}{3}, -\frac{1}{3})$ and $(3-3.8,\ 3-3.8) = (-0.8, -0.8)$.
  $sim = \dfrac{(-\frac{1}{3})(-0.8) + (-\frac{1}{3})(-0.8)}{\sqrt{\frac{1}{9}+\frac{1}{9}}\,\sqrt{0.64+0.64}} = \dfrac{0.5333}{0.4714 \times 1.1314} = \dfrac{0.5333}{0.5333} = 1.$
- **User 5**, $S = \{3,6\}$: deviations $(\frac{2}{3}, -\frac{1}{3})$ and $(2-3.5,\ 3-3.5) = (-1.5, -0.5)$.
  $sim = \dfrac{\frac{2}{3}(-1.5) + (-\frac{1}{3})(-0.5)}{\sqrt{\frac{4}{9}+\frac{1}{9}}\,\sqrt{2.25+0.25}} = \dfrac{-1+0.1667}{0.7454 \times 1.5811} = \dfrac{-0.8333}{1.1785} = -0.707$ (exactly $-1/\sqrt{2}$).
- **User 8**, $S = \{6\}$: deviations $(-\frac{1}{3})$ and $(2-3) = (-1)$. $\quad sim = \dfrac{(-\frac{1}{3})(-1)}{\sqrt{1/9}\,\sqrt{1}} = \dfrac{1/3}{1/3} = 1.$
- **User 11**, $S = \{1,3,6\}$: deviations $(-\frac{1}{3}, \frac{2}{3}, -\frac{1}{3})$ and $(4-3,\ 5-3,\ 4-3) = (1, 2, 1)$.
  $sim = \dfrac{-\frac{1}{3} + \frac{4}{3} - \frac{1}{3}}{\sqrt{\frac{1}{9}+\frac{4}{9}+\frac{1}{9}}\,\sqrt{1+4+1}} = \dfrac{2/3}{\sqrt{6}/3 \cdot \sqrt{6}} = \dfrac{2/3}{2} = \dfrac{1}{3} \approx 0.333.$

Answer. $sim(1,2) = 1$, $sim(1,3) = 1$, $sim(1,5) = -1/\sqrt{2} \approx -0.707$, $sim(1,8) = 1$, $sim(1,11) = 1/3 \approx 0.333$.

(b) Sorting by similarity: Users 2, 3, 8 (all 1), then User 11 (0.333), then User 5 (−0.707). The three-way tie at the top does not matter, because all three fit into N and the fourth value is strictly smaller.

Answer. $N = \{\text{User 2}, \text{ User 3}, \text{ User 8}\}$, each with similarity 1.

(c) The three neighbors rated Movie 4 with 2, 4 and 4 stars:

$$\hat{r}_{1,4} = \frac{1 \cdot 2 + 1 \cdot 4 + 1 \cdot 4}{1 + 1 + 1} = \frac{10}{3} \approx 3.33.$$

Because all three weights are equal, the similarity-weighted average coincides here with the plain average (Option 1 of the lecture).

Answer. $\hat{r}_{1,4} = 10/3 \approx 3.33$ stars.

(d) Overlap with User 1. From the table: User 2 shares *one* movie (Movie 3), User 8 shares *one* movie (Movie 6), and User 3 shares *two* (Movies 1 and 6). User 11, who shares all three of User 1's movies, is not in N.

One co-rated movie gives ±1. Let $S_{xy} = \{s\}$ and write $a = r_{xs} - \bar{r}_x$, $b = r_{ys} - \bar{r}_y$. Each sum has one term, so

$$sim(x, y) = \frac{ab}{\sqrt{a^2}\sqrt{b^2}} = \frac{ab}{|a|\,|b|} = \begin{cases} +1 & \text{if } a \text{ and } b \text{ have the same sign,} \\ -1 & \text{if they have opposite signs,} \end{cases}$$

and it is undefined (0/0) if $a = 0$ or $b = 0$. Such a similarity carries one bit of information: whether both users rated that single movie on the same side of their own average. The value 1 for User 3 is equally weak: both users rated Movies 1 and 6 identically (1, 1 and 3, 3), so both deviation vectors are multiples of $(1, 1)$ and their cosine is automatically ±1 (here +1, because both pairs lie below the user's mean).

Trust. Low. The three "perfect" neighbors are perfect only because they share one or two movies with User 1, and they disagree about Movie 4 (2, 4 and 4 stars). User 11 has the most evidence (three shared movies) but a much lower similarity and a rating of 2; if every user with a positive similarity is used (Users 2, 3, 8, 11) the prediction becomes $(2 + 4 + 4 + \frac{1}{3} \cdot 2)/(3 + \frac{1}{3}) = 3.2$. The prediction is therefore sensitive to which few ratings happen to overlap.

Drawback. This is the *sparsity* problem of collaborative filtering: the matrix is sparse, so it is hard to find users who rated the same items, and similarities computed from a handful of co-rated items are unreliable. User 1 has also rated only three movies, a mild form of the cold-start problem. Item-item CF (next exercise) is often more reliable, because items usually have many more ratings than users. (A common remedy, beyond the lecture, is to shrink similarities that rest on few co-rated items.)

Exercise 3: Item-Item CF with Baselines

Ex3: Item-Item CF with Baselines

Five users (A–E) have rated four movies ($M1$–$M4$) on a 1–5 star scale; "–" means unknown. Rows are users and columns are movies, as in the lecture's utility matrix:

	$M1$	$M2$	$M3$	$M4$
A	–	3	3	2
B	3	1	–	2
C	5	3	2	4
D	4	–	4	–
E	–	1	3	2

We want to predict $\hat{r}_{E,M1}$, user E's rating of movie $M1$, with item-item collaborative filtering and a neighborhood of $k = 2$ movies.

The similarity s_{ij} of movies i and j is the *centered cosine*: subtract from each column the mean of its known ratings, set the unknown entries to 0, and take the cosine of the two resulting columns.

(a) Compute $s_{M1,M2}$, $s_{M1,M3}$ and $s_{M1,M4}$.

(b) Find $N(M1;E)$, the $k = 2$ movies rated by E that are most similar to $M1$, and predict $\hat{r}_{E,M1} = \sum_{j \in N(M1;E)} s_{M1,j}\, r_{E,j} \big/ \sum_{j \in N(M1;E)} s_{M1,j}$.

(c) Now use the baseline estimate $b_{xi} = \mu + b_x + b_i$, where μ is the mean of all 15 known ratings, b_x = (mean rating of user x) $- \mu$ and b_i = (mean rating of movie i) $- \mu$. Predict

$$\hat{r}_{E,M1} = b_{E,M1} + \frac{\sum_{j \in N(M1;E)} s_{M1,j}\,(r_{E,j} - b_{E,j})}{\sum_{j \in N(M1;E)} s_{M1,j}}$$

and explain why the result differs from (b).

(d) On Netflix there are $|X| \approx 480{,}000$ users and $|S| = 17{,}770$ movies. Taking one similarity evaluation as the elementary operation, how many operations does it take to precompute the similarities of all pairs of users, and of all pairs of movies? By roughly what factor do the two counts differ? Give one more reason why item-item CF is often preferred in practice.

Solution

(a) Step 1: column means. Each movie's mean is taken over its known ratings only:

$$m_1 = \tfrac{3+5+4}{3} = 4, \qquad m_2 = \tfrac{3+1+3+1}{4} = 2, \qquad m_3 = \tfrac{3+2+4+3}{4} = 3, \qquad m_4 = \tfrac{2+2+4+2}{4} = 2.5.$$

Step 2: centered columns (entries in the user order A, B, C, D, E; an unknown rating becomes 0, i.e. "average"):

	A	B	C	D	E	squared length
c_1 ($M1$)	0	−1	1	0	0	2
c_2 ($M2$)	1	−1	1	0	−1	4
c_3 ($M3$)	0	0	−1	1	0	2
c_4 ($M4$)	−0.5	−0.5	1.5	0	−0.5	3

(For example, D rated $M1$ exactly at its mean, so that entry is also 0.)

Step 3: cosines. Only users with nonzero entries in *both* columns contribute to a dot product:

$$s_{M1,M2} = \frac{c_1 \cdot c_2}{\|c_1\|\,\|c_2\|} = \frac{(-1)(-1)+(1)(1)}{\sqrt{2}\cdot\sqrt{4}} = \frac{2}{2\sqrt{2}} = \frac{1}{\sqrt{2}} \approx 0.707,$$
$$s_{M1,M3} = \frac{(1)(-1)}{\sqrt{2}\cdot\sqrt{2}} = -\frac{1}{2} = -0.5,$$
$$s_{M1,M4} = \frac{(-1)(-0.5)+(1)(1.5)}{\sqrt{2}\cdot\sqrt{3}} = \frac{2}{\sqrt{6}} \approx 0.816.$$

Answer. $s_{M1,M2} \approx 0.707$, $s_{M1,M3} = -0.5$, $s_{M1,M4} \approx 0.816$.

(b) User E rated $M2$, $M3$ and $M4$. The two most similar to $M1$ are $M4$ (0.816) and $M2$ (0.707); $M3$ has a negative similarity and is not used. E rated them $r_{E,M4} = 2$ and $r_{E,M2} = 1$:

$$\hat{r}_{E,M1} = \frac{0.816 \cdot 2 + 0.707 \cdot 1}{0.816 + 0.707} = \frac{1.633 + 0.707}{1.523} = \frac{2.340}{1.523} \approx 1.54.$$

Answer. $N(M1; E) = \{M4, M2\}$ and $\hat{r}_{E,M1} \approx 1.54$ stars.

(c) Step 1: baseline quantities. The 15 known ratings sum to $8+6+14+8+6 = 42$ (row sums A to E), so

$$\mu = 42/15 = 2.8.$$

User E's mean is $(1+3+2)/3 = 2$, so $b_E = 2 - 2.8 = -0.8$ (E is a harsh rater). The movie biases follow from the column means of part (a):

$$b_{M1} = 4 - 2.8 = 1.2, \qquad b_{M2} = 2 - 2.8 = -0.8, \qquad b_{M4} = 2.5 - 2.8 = -0.3$$

($b_{M3} = 0.2$ is not needed).

Step 2: baselines of the three cells involved.

$$b_{E,M1} = 2.8 - 0.8 + 1.2 = 3.2, \qquad b_{E,M2} = 2.8 - 0.8 - 0.8 = 1.2, \qquad b_{E,M4} = 2.8 - 0.8 - 0.3 = 1.7.$$

Step 3: deviations of E's neighbor ratings from their baselines.

$$r_{E,M2} - b_{E,M2} = 1 - 1.2 = -0.2, \qquad r_{E,M4} - b_{E,M4} = 2 - 1.7 = +0.3.$$

Step 4: prediction.

$$\hat{r}_{E,M1} = 3.2 + \frac{0.816 \cdot 0.3 + 0.707 \cdot (-0.2)}{0.816 + 0.707} = 3.2 + \frac{0.2449 - 0.1414}{1.523} = 3.2 + 0.068 \approx 3.27.$$

Answer. $\mu = 2.8$, $b_E = -0.8$, $b_{M1} = 1.2$, $b_{E,M1} = 3.2$, and $\hat{r}_{E,M1} \approx 3.27$ stars (versus 1.54 in (b)).

Why the two predictions differ. Prediction (b) copies E's raw ratings of $M4$ and $M2$. But those movies are rated low by *everyone* (means 2.5 and 2), whereas $M1$ is the best-liked movie (mean 4). The users who rated $M1$ together with $M2$ and $M4$ confirm this: B gave 3 versus $1, 2$, and C gave 5 versus $3, 4$. So E's low ratings mostly reflect that $M2$ and $M4$ are rated low by everyone, not a dislike of movies like $M1$. The baseline version asks how E's ratings *deviate from what was expected*: -0.2 and $+0.3$, i.e. essentially as expected. It therefore predicts close to E's baseline for $M1$. In the lecture's words, the neighbors now contribute only what is surprising about their ratings.

Algebraically, because the weights are normalized, μ and b_E cancel from the correction and

$$\hat{r}^{(c)} - \hat{r}^{(b)} = b_{M1} - \frac{0.816\, b_{M4} + 0.707\, b_{M2}}{0.816 + 0.707} = 1.2 - (-0.532) = 1.732,$$

which is exactly $3.268 - 1.536$: the correction is how much better $M1$ is liked than its neighbors are on average.

(d) Counting. Precomputing all pairwise similarities needs one evaluation per unordered pair, i.e. $\binom{|X|}{2}$ evaluations for the users and $\binom{|S|}{2}$ for the movies:

$$\binom{|X|}{2} = \tfrac{1}{2}\,(480{,}000 \cdot 479{,}999) = 115{,}199{,}760{,}000 \approx 1.15 \times 10^{11},$$
$$\binom{|S|}{2} = \tfrac{1}{2}\,(17{,}770 \cdot 17{,}769) = 157{,}877{,}565 \approx 1.58 \times 10^{8}.$$

Both counts are quadratic, $\Theta(|X|^2)$ and $\Theta(|S|^2)$, so their ratio is about

$$(|X|/|S|)^2 = (480{,}000/17{,}770)^2 \approx 27^2 \approx 730.$$

(Each evaluation itself costs time proportional to the number of co-rated entries; we count evaluations, as instructed.)

Answer. About 1.15×10^{11} user-user similarities versus 1.58×10^{8} item-item similarities: precomputing for items is roughly 730 times cheaper.

Another reason. Items are "simpler" than users: a movie's rating pattern is stable over time, while users have several tastes that change. Precomputed item similarities therefore stay valid much longer and can be refreshed rarely. They are also more reliable: on Netflix a movie has on average $10^8/17{,}770 \approx 5{,}600$ ratings but a user only $10^8/480{,}000 \approx 210$, so two movies typically share many more raters than two users share movies (compare Exercise 2). The lecture notes that in practice item-item often works better than user-user.

Exercise 4: Evaluating Recommendations

Ex4: Evaluating Recommendations

Five of a user's ratings were hidden before training (a held-out set). Two recommenders, A and B, were trained on the remaining data; their predictions for the five hidden ratings are:

Movie	$M1$	$M2$	$M3$	$M4$	$M5$
True rating r_{xi}	3	5	1	4	2
Prediction of A	4	3.5	2.5	3	2
Prediction of B	1	5	3.5	4.5	4

A movie is *relevant* to the user if its true rating is at least 4.

(a) Compute the RMSE of each recommender on the five held-out ratings,

$$RMSE = \sqrt{\frac{1}{|R|}\sum_{(x,i)\in R}(\hat{r}_{xi} - r_{xi})^2},$$

where R is the held-out set. Which recommender is more accurate by RMSE?

(b) Each recommender shows the user its top-k of these five movies, ranked by predicted rating. Compute precision@k = (number of relevant movies among the top k)$/k$ for both recommenders, for $k = 2$ and $k = 3$.

(c) Which recommender would you deploy for a "Top 2 for you" row, and why? Relate your answer to the lecture's "Problems with Error Measures".

(d) The Netflix Prize required a 10% improvement in RMSE over Cinematch, which scored 0.9514 on the quiz set. What RMSE is 10% lower? By what percentage must the sum of squared errors (SSE) on the same quiz set fall?

Solution

(a) Here $|R| = 5$. The errors $\hat{r}_{xi} - r_{xi}$ and their squares:

Movie	$M1$	$M2$	$M3$	$M4$	$M5$	SSE
error of A	+1	−1.5	+1.5	−1	0	
squared	1	2.25	2.25	1	0	6.5
error of B	−2	0	+2.5	+0.5	+2	
squared	4	0	6.25	0.25	4	14.5

$$RMSE_A = \sqrt{6.5/5} = \sqrt{1.3} \approx 1.140, \qquad RMSE_B = \sqrt{14.5/5} = \sqrt{2.9} \approx 1.703.$$

Answer. $RMSE_A \approx 1.14$ and $RMSE_B \approx 1.70$: by RMSE, A is clearly more accurate.

(b) The relevant movies are $M2$ (5 stars) and $M4$ (4 stars). Sorting each recommender's predictions from highest to lowest (there are no ties):

- A: $M1$ (4), $M2$ (3.5), $M4$ (3), $M3$ (2.5), $M5$ (2);
- B: $M2$ (5), $M4$ (4.5), $M5$ (4), $M3$ (3.5), $M1$ (1).

	top 2	precision@2	top 3	precision@3
A	$M1, M2$	$1/2$	$M1, M2, M4$	$2/3$
B	$M2, M4$	$2/2 = 1$	$M2, M4, M5$	$2/3$

Answer. precision@2: $A = 0.5$, $B = 1$. precision@3: $A = B = 2/3$.

(c) Deploy B. Its top 2 contains exactly the two movies the user loves, while A leads with $M1$, a movie the user rated only 3. B's large errors are on $M1$, $M3$ and $M5$, which the user dislikes or is lukewarm about. On $M1$, B even errs in the harmless direction (it predicts 1, so $M1$ is never shown), and $M3$ and $M5$, though overrated, still rank below the relevant movies. RMSE weighs every held-out cell equally, so it penalizes B for errors at the bottom of the list that the user never sees.

This is the lecture's point: "in practice, we care only to predict high ratings: RMSE might penalize a method that does well for high ratings and badly for others", and "RMSE weighs every held-out cell equally, but only the top of the list is ever seen".

Two caveats. First, the verdict depends on how the list is shown: at $k = 3$ both recommenders score $2/3$, because B's overrating of $M5$ (predicted 4, true 2) now reaches the list. The metric should match the way recommendations are displayed. Second, five ratings of one user are far too few to decide; in practice the metrics are averaged over many users and held-out ratings.

(d) The target is $0.9 \times 0.9514 = 0.85626 \approx 0.8563$, the quiz-set bar quoted in the lecture. (The prize itself was decided on the hidden test half, where Cinematch scored 0.9525 and the bar was 0.8572; the percentages below are the same.) On a fixed quiz set, $SSE = |R| \cdot RMSE^2$ with $|R|$ fixed, so

$$\frac{SSE_{\text{new}}}{SSE_{\text{old}}} = \left(\frac{RMSE_{\text{new}}}{RMSE_{\text{old}}}\right)^2 = 0.9^2 = 0.81.$$

Answer. Target RMSE ≈ 0.8563; the SSE must fall by $1 - 0.81 = 19\%$.

(Because RMSE and SSE are monotonically related, minimizing one minimizes the other, as the lecture notes; a "10%" improvement in RMSE is a 19% improvement in squared error.)

Exercise 5: Learned Interpolation Weights

Ex5: Learned Interpolation Weights

Recall the model with learned interpolation weights,

$$\hat{r}_{xi} = b_{xi} + \sum_{j \in N(i;x)} w_{ij}\,(r_{xj} - b_{xj}), \qquad J(w) = \sum_{(x,i)\in R} \big(\hat{r}_{xi} - r_{xi}\big)^2,$$

where b_{xi} is the baseline estimate of r_{xi} and the weights w minimize the training error $J(w)$.

Fix a target movie i with two neighbors j and k. Four users $x = 1, 2, 3, 4$ rated all three movies, $N(i;x) = \{j, k\}$ for each of them, and no other user rated i, so w_{ij} and w_{ik} appear in exactly four terms of J. Write $d_{xm} = r_{xm} - b_{xm}$ for the deviation of a rating from its baseline:

User x	d_{xj}	d_{xk}	d_{xi}
1	2	1	2
2	1	1	2
3	−1	−1	−1
4	−1	1	0.5

(a) Show that $\hat{r}_{xi} - r_{xi} = w_{ij}\,d_{xj} + w_{ik}\,d_{xk} - d_{xi}$, and write the four terms of J that involve w_{ij}, w_{ik} as a function $J(w_{ij}, w_{ik})$. Evaluate it at the similarity weights $w_{ij} = 0.6$, $w_{ik} = 0.4$ (normalized to sum to 1, as in a similarity-weighted average).

(b) Using the lecture's gradient

$$\frac{\partial J}{\partial w_{ij}} = 2\sum_{x} \Big(\Big[b_{xi} + \sum_{m\in N(i;x)} w_{im}(r_{xm} - b_{xm})\Big] - r_{xi}\Big)(r_{xj} - b_{xj}),$$

compute ∇J at $w = (w_{ij}, w_{ik}) = (0, 0)$ and take one gradient-descent step $w \leftarrow w - \eta \nabla J$ with $\eta = 0.05$. Evaluate J before and after the step.

(c) Find the minimizer $w^* = (w^*_{ij}, w^*_{ik})$ by setting both partial derivatives to zero. Compute $J(w^*)$ and compare it with (a) and (b). Do the learned weights sum to 1?

(d) Suppose instead that movie k is an exact duplicate of j ($d_{xk} = d_{xj}$ for every user x). Show that J then depends on w_{ij} and w_{ik} only through $w_{ij} + w_{ik}$. How do learned weights treat the duplicate, compared with similarity weights?

Solution

(a) The error of one prediction. With $N(i;x) = \{j, k\}$,

$$\hat{r}_{xi} - r_{xi} = b_{xi} + w_{ij}(r_{xj} - b_{xj}) + w_{ik}(r_{xk} - b_{xk}) - r_{xi} = w_{ij}\,d_{xj} + w_{ik}\,d_{xk} - \underbrace{(r_{xi} - b_{xi})}_{d_{xi}}.$$

The baseline drops out: the weights only have to explain the target's *deviation* from its baseline using the neighbors' deviations. We write $e_x = w_{ij}d_{xj} + w_{ik}d_{xk} - d_{xi}$.

The function J. Substituting the table, row by row:

$$J(w_{ij}, w_{ik}) = \underbrace{(2w_{ij} + w_{ik} - 2)^2}_{x=1} + \underbrace{(w_{ij} + w_{ik} - 2)^2}_{x=2} + \underbrace{(-w_{ij} - w_{ik} + 1)^2}_{x=3} + \underbrace{(-w_{ij} + w_{ik} - 0.5)^2}_{x=4}.$$

At the similarity weights $(0.6, 0.4)$**:**

$e_1 = 1.2+0.4-2 = -0.4$, $e_2 = 0.6+0.4-2 = -1$, $e_3 = -0.6-0.4+1 = 0$, $e_4 = -0.6+0.4-0.5 = -0.7$,

so

$$J(0.6, 0.4) = 0.16 + 1 + 0 + 0.49 = 1.65.$$

Answer. $\hat{r}_{xi} - r_{xi} = w_{ij}d_{xj} + w_{ik}d_{xk} - d_{xi}$; $J(0.6, 0.4) = 1.65$.

(b) The gradient in this setting. The bracket in the lecture's formula is $\hat{r}_{xi}$, so the factor is e_x; every user has j (and k) among the neighbors, so

$$\frac{\partial J}{\partial w_{ij}} = 2\sum_{x=1}^{4} e_x\, d_{xj}, \qquad \frac{\partial J}{\partial w_{ik}} = 2\sum_{x=1}^{4} e_x\, d_{xk}.$$

At $w = (0, 0)$ every prediction equals the baseline, so $e_x = -d_{xi}$, i.e. $e = (-2, -2, 1, -0.5)$:

$$\frac{\partial J}{\partial w_{ij}} = 2\big[(-2)(2) + (-2)(1) + (1)(-1) + (-0.5)(-1)\big] = 2(-6.5) = -13,$$
$$\frac{\partial J}{\partial w_{ik}} = 2\big[(-2)(1) + (-2)(1) + (1)(-1) + (-0.5)(1)\big] = 2(-5.5) = -11.$$

One step: $w \leftarrow (0, 0) - 0.05\,(-13, -11) = (0.65,\ 0.55)$.

J **before and after.** $J(0, 0) = \sum_x d_{xi}^2 = 4 + 4 + 1 + 0.25 = 9.25$. At $(0.65, 0.55)$:

$$e = (1.3 + 0.55 - 2,\ \ 0.65 + 0.55 - 2,\ \ -1.2 + 1,\ \ -0.65 + 0.55 - 0.5) = (-0.15,\ -0.8,\ -0.2,\ -0.6),$$

so

$$J(0.65, 0.55) = 0.0225 + 0.64 + 0.04 + 0.36 = 1.0625.$$

Answer. $\nabla J(0, 0) = (-13, -11)$; after one step $w = (0.65, 0.55)$, and J falls from 9.25 to 1.0625, already below the 1.65 of the similarity weights.

(c) Normal equations. Setting $\sum_x e_x d_{xj} = 0$ and $\sum_x e_x d_{xk} = 0$ and expanding e_x gives two linear equations:

$$\Big(\sum_x d_{xj}^2\Big) w_{ij} + \Big(\sum_x d_{xj}d_{xk}\Big) w_{ik} = \sum_x d_{xj}d_{xi},$$
$$\Big(\sum_x d_{xj}d_{xk}\Big) w_{ij} + \Big(\sum_x d_{xk}^2\Big) w_{ik} = \sum_x d_{xk}d_{xi}.$$

From the table: $\sum d_{xj}^2 = 4 + 1 + 1 + 1 = 7$, $\sum d_{xk}^2 = 4$, $\sum d_{xj}d_{xk} = 2 + 1 + 1 - 1 = 3$, $\sum d_{xj}d_{xi} = 4 + 2 + 1 - 0.5 = 6.5$, $\sum d_{xk}d_{xi} = 2 + 2 + 1 + 0.5 = 5.5$. Hence

$$7w_{ij} + 3w_{ik} = 6.5, \qquad 3w_{ij} + 4w_{ik} = 5.5.$$

The determinant is $7 \cdot 4 - 3 \cdot 3 = 19$, and Cramer's rule gives

$$w_{ij}^* = \frac{4(6.5) - 3(5.5)}{19} = \frac{9.5}{19} = 0.5, \qquad w_{ik}^* = \frac{7(5.5) - 3(6.5)}{19} = \frac{19}{19} = 1.$$

Check: $7(0.5) + 3(1) = 6.5$ and $3(0.5) + 4(1) = 5.5$. This stationary point is the global minimum: J is a sum of squares of affine functions, i.e. a convex quadratic, and its Hessian

$$\nabla^2 J = 2 \begin{bmatrix} 7 & 3 \\ 3 & 4 \end{bmatrix}$$

is positive definite (positive trace and determinant).

$J(w^*)$**:**

$$e = (1{+}1{-}2,\ 0.5{+}1{-}2,\ -0.5{-}1{+}1,\ -0.5{+}1{-}0.5) = (0,\ -0.5,\ -0.5,\ 0), \quad J(w^*) = 0.25{+}0.25 = 0.5.$$

Answer. $w^* = (0.5,\ 1)$ with $J(w^*) = 0.5$. The sequence $J = 9.25$ (no neighbors) $\to$ 1.65 (similarity weights) $\to$ 1.0625 (one GD step) $\to$ 0.5 (optimum) shows the learned weights fit the training data about three times better than the similarity weights.

Sum of the weights. $w_{ij}^* + w_{ik}^* = 1.5 \neq 1$. In these data the target deviates more strongly than its neighbors (e.g. user 2: neighbors +1, +1, target +2), which a weighted *average* (weights summing to 1) cannot express. This is restriction 3) on the lecture's "Limits of Similarity Weights" slide. Note also that $w_{ik}^* > w_{ij}^*$ even though the similarity weights rank j first: learned weights reflect each neighbor's role in the prediction *given the other neighbor*, not a pairwise similarity.

(A remark on (b): the Hessian's eigenvalues are about 4.29 and 17.71, so gradient descent on this J converges for any $\eta < 2/17.71 \approx 0.113$; $\eta = 0.05$ is safe.)

(d) If $d_{xk} = d_{xj}$ for all x, then

$$e_x = w_{ij} d_{xj} + w_{ik} d_{xj} - d_{xi} = (w_{ij} + w_{ik})\, d_{xj} - d_{xi},$$

so every term of J, and hence J, depends on the two weights only through $W = w_{ij} + w_{ik}$. Minimizing the one-variable quadratic $\sum_x (W d_{xj} - d_{xi})^2$ gives $W^* = \sum_x d_{xj} d_{xi} \big/ \sum_x d_{xj}^2 = 6.5/7 \approx 0.93$, the same total weight that j alone would get. Any split with $w_{ij} + w_{ik} = W^*$ is optimal (the normal equations become singular, with determinant $7 \cdot 7 - 7 \cdot 7 = 0$).

Answer. The duplicate does not add influence: the two copies *share* one total weight. With similarity weights, $s_{ik} = s_{ij}$, so each copy receives a full vote and the same evidence counts twice. For example, with a third neighbor l of equal similarity, the duplicated movie would get 2/3 of the weight instead of 1/2. This is restriction 2) of the lecture.

Exercise 6: Latent Factor Interpretation

Ex6: Latent Factor Interpretation

Suppose a latent factor model with two factors has been trained, and the two factors are interpreted as the genres Comedy and Action. After training, you have the following factor vectors for users and movies:

- User factors p_x (Comedy, Action): U1 (0.1, 0.9), U2 (0.8, 0.2)
- Movie factors q_i (Comedy, Action): M1 (0.9, 0.1), M2 (0.2, 0.8), M3 (0.5, 0.5)

The model predicts $\hat{r}_{xi} = q_i \cdot p_x$. It has no bias terms, so these scores rank the movies for a user but are not on a star scale.

(a) For each user, compute the predicted score of each movie (show the dot product calculations), and predict which movie each user is likely to rate highest.

(b) A second training run on the same data returns rotated factors $p'_x = p_x O$ and $q'_i = q_i O$ (row vectors), where

$$O = \begin{bmatrix} 0.6 & -0.8 \\ 0.8 & 0.6 \end{bmatrix}, \qquad O\,O^T = I.$$

Compute p'_{U1}, q'_{M2} and $q'_{M2} \cdot p'_{U1}$. Show that every prediction is unchanged. What does this imply about naming the factors "Comedy" and "Action"?

Solution

(a) The dot product multiplies matching coordinates (Comedy with Comedy, Action with Action) and adds:

	User U1 = (0.1, 0.9)	User U2 = (0.8, 0.2)
M1 (0.9, 0.1)	$0.1 \cdot 0.9 + 0.9 \cdot 0.1 = 0.09 + 0.09 = 0.18$	$0.8 \cdot 0.9 + 0.2 \cdot 0.1 = 0.72 + 0.02 = \mathbf{0.74}$
M2 (0.2, 0.8)	$0.1 \cdot 0.2 + 0.9 \cdot 0.8 = 0.02 + 0.72 = \mathbf{0.74}$	$0.8 \cdot 0.2 + 0.2 \cdot 0.8 = 0.16 + 0.16 = 0.32$
M3 (0.5, 0.5)	$0.1 \cdot 0.5 + 0.9 \cdot 0.5 = 0.05 + 0.45 = 0.50$	$0.8 \cdot 0.5 + 0.2 \cdot 0.5 = 0.40 + 0.10 = 0.50$

Answer. U1 ranks M2 (0.74) > M3 (0.50) > M1 (0.18), so U1 is likely to rate **M2** highest. U2 ranks M1 (0.74) > M3 (0.50) > M2 (0.32), so U2 is likely to rate **M1** highest.

This matches the interpretation: U1 leans strongly to Action and M2 is the Action movie; U2 leans to Comedy and M1 is the Comedy movie. The balanced movie M3 scores 0.5 for both users, in the middle, since $0.5\,(p_{x1} + p_{x2}) = 0.5$ whenever a user's two coordinates sum to 1.

(b) Rotated vectors. A row vector times O: $(a, b)\,O = (0.6a + 0.8b,\ -0.8a + 0.6b)$.

$$p'_{U1} = (0.6 \cdot 0.1 + 0.8 \cdot 0.9,\ -0.8 \cdot 0.1 + 0.6 \cdot 0.9) = (0.78,\ 0.46),$$

$$q'_{M2} = (0.6 \cdot 0.2 + 0.8 \cdot 0.8,\ -0.8 \cdot 0.2 + 0.6 \cdot 0.8) = (0.76,\ 0.32),$$

$$q'_{M2} \cdot p'_{U1} = 0.76 \cdot 0.78 + 0.32 \cdot 0.46 = 0.5928 + 0.1472 = 0.74 = \hat{r}_{U1,M2}.$$

Every prediction is unchanged. Writing the dot product as a matrix product of row vectors and using $O\,O^T = I$:

$$q_i' \cdot p_x' = (q_i O)(p_x O)^T = q_i\, O\, O^T\, p_x^T = q_i\, p_x^T = q_i \cdot p_x.$$

For instance, with $p_{U2}' = (0.64, -0.52)$, $q_{M1}' = (0.62, -0.66)$ and $q_{M3}' = (0.70, -0.10)$, all six rotated predictions reproduce the table of (a) exactly.

Answer. $p_{U1}' = (0.78, 0.46)$, $q_{M2}' = (0.76, 0.32)$ and $q_{M2}' \cdot p_{U1}' = 0.74$, as before; since $O\,O^T = I$, all predictions are invariant.

What this implies. Both sets of factors make identical predictions, so they fit the ratings equally well. The penalty terms of a regularized model are unchanged too, because $\|p_x O\|^2 = p_x O O^T p_x^T = \|p_x\|^2$. The data therefore cannot prefer one coordinate system over the other. In the rotated system the first coordinate of a user is $0.6 \cdot \text{Comedy} + 0.8 \cdot \text{Action}$, a mixture with no genre name. The labels "Comedy" and "Action" are a human interpretation attached *after* seeing which movies score high on each axis. The algorithm returns unlabeled coordinates, and another run may return different but equally valid axes, as the lecture's latent-factor slide points out. Only quantities that do not depend on the rotation, such as predictions, vector lengths and angles between vectors, are meaningful.

Exercise 7: Matrix Factorization Optimization

Ex7: Matrix Factorization Optimization

Given a small user-item rating matrix ("?" = unknown):

	$M1$	$M2$	$M3$
$U1$	4	?	2
$U2$	3	3	?
$U3$	?	1	4

We fit a latent factor model with $k = 2$ factors, $\hat{r}_{xi} = q_i \cdot p_x$, where p_x (row x of P) is user x's factor vector and q_i (row i of Q) is movie i's. The matrices P and Q are initialized with small random numbers; use the values

$$P_0 = \begin{bmatrix} 0.3 & 0.1 \\ 0.2 & 0.2 \\ 0.1 & 0.4 \end{bmatrix} \begin{matrix} \leftarrow U1 \\ \leftarrow U2 \\ \leftarrow U3 \end{matrix} \qquad Q_0 = \begin{bmatrix} 0.4 & 0.1 \\ 0.2 & 0.3 \\ 0.1 & 0.5 \end{bmatrix} \begin{matrix} \leftarrow M1 \\ \leftarrow M2 \\ \leftarrow M3 \end{matrix}$$

and minimize the regularized objective from the lecture, where R is the set of known ratings:

$$\min_{P,Q} \sum_{(x,i)\in R} (r_{xi} - q_i \cdot p_x)^2 + \lambda_1 \sum_x \|p_x\|^2 + \lambda_2 \sum_i \|q_i\|^2.$$

Use the learning rate $\eta = 0.01$ and the regularization parameters $\lambda_1 = \lambda_2 = 0.1$. One iteration of gradient descent computes, from the current P and Q,

$$\nabla p_{xf} = \sum_{i:\,(x,i)\in R} -2(r_{xi} - q_i \cdot p_x)\, q_{if} + 2\lambda_1 p_{xf},$$
$$\nabla q_{if} = \sum_{x:\,(x,i)\in R} -2(r_{xi} - q_i \cdot p_x)\, p_{xf} + 2\lambda_2 q_{if},$$

for every factor f, and then updates $P \leftarrow P - \eta \nabla P$ and $Q \leftarrow Q - \eta \nabla Q$.

(a) Which ratings appear in the objective? A classmate proposes to fill in the three missing ratings first – with the average known rating ($17/6 \approx 2.83$), or with 0 as plain SVD would – and then fit all nine entries. Explain why the objective sums over the known ratings only.

(b) Perform one iteration of gradient descent: compute the six errors $e_{xi} = r_{xi} - q_i \cdot p_x$, the gradients, and the updated matrices P_1 and Q_1.

(c) Compute the effect of regularization on the first row p_{U1} of P: repeat its update with $\lambda_1 = 0.5$ (all else unchanged) and compare with (b). Show that the update can be written as $p_{U1} \leftarrow (1 - 2\eta\lambda_1)\, p_{U1} + 2\eta \sum_i e_{U1,i}\, q_i$, and explain how increasing λ_1 affects the values in P, after one iteration and after many. Does λ_1 change Q_1?

(d) Stochastic gradient descent (SGD) instead makes a step after each single rating r_{xi}. With the lecture's update rule (learning rate η; both updates computed from the old q_i and p_x),

$$\begin{aligned} \xi_{xi} &= 2(r_{xi} - q_i \cdot p_x), \\ q_i &\leftarrow q_i + \eta\,(\xi_{xi}\, p_x - \lambda_2\, q_i), \\ p_x &\leftarrow p_x + \eta\,(\xi_{xi}\, q_i - \lambda_1\, p_x), \end{aligned}$$

perform one SGD step on the rating $r_{U1,M1} = 4$, starting from P_0 and Q_0 (with $\eta = 0.01$ and $\lambda_1 = \lambda_2 = 0.1$).

How many known ratings does one gradient-descent iteration process, and how many does one SGD step process? With k factors, about how many arithmetic operations does each need? What does this mean on Netflix, with $|R| \approx 10^8$ known ratings?

Solution

Setting up. The known ratings are

$$r_{U1,M1} = 4, \quad r_{U1,M3} = 2, \quad r_{U2,M1} = 3, \quad r_{U2,M2} = 3, \quad r_{U3,M2} = 1, \quad r_{U3,M3} = 4,$$

so $|R| = 6$. The model has $3 \cdot 2 + 3 \cdot 2 = 12$ parameters. In vector form the gradients read

$$\nabla p_x = -2 \sum_{i:(x,i)\in R} e_{xi}\, q_i + 2\lambda_1 p_x, \qquad \nabla q_i = -2 \sum_{x:(x,i)\in R} e_{xi}\, p_x + 2\lambda_2 q_i, \qquad e_{xi} = r_{xi} - q_i \cdot p_x.$$

(a) Only the six known ratings appear in the error term, together with the two penalty terms; the three "?" cells do not appear at all.

Filling with 0 (what a plain SVD of the matrix implicitly does) declares that the user gave the movie 0 stars, below the lowest real rating. The fit would then try to reproduce these fake zeros and drag the factors, and so every prediction, towards 0. This is the complication pointed out on the lecture's SVD recap slide.

Filling with 17/6 is milder but still invents data. A third of the nine training entries would be fabricated, and the model would be rewarded for predicting 2.83 in exactly the three cells we want to predict. The "predictions" would then largely echo the fill value instead of what the known ratings imply. Real matrices make this much worse: on Netflix about 8.5×10^9 cells contain only 10^8 ratings, so fabricated entries would outnumber real ones by about 84 to 1 and dominate the fit. Fitting every cell would also cost time proportional to $|X|\,|S|$ instead of $|R|$.

Answer. The objective contains only the six known ratings. A missing rating is unknown, not a rating of 0 or of 17/6, and the factors should be fitted to observed information only. (The average 17/6 does have a proper role: it is the global mean μ of the model with biases; see Exercise 8(c).)

(b) Step 1: predictions and errors with P_0, Q_0.

(x,i)	$q_i \cdot p_x$	r_{xi}	e_{xi}
$(U1,M1)$	$0.4 \cdot 0.3 + 0.1 \cdot 0.1 = 0.13$	4	3.87
$(U1,M3)$	$0.1 \cdot 0.3 + 0.5 \cdot 0.1 = 0.08$	2	1.92
$(U2,M1)$	$0.4 \cdot 0.2 + 0.1 \cdot 0.2 = 0.10$	3	2.90
$(U2,M2)$	$0.2 \cdot 0.2 + 0.3 \cdot 0.2 = 0.10$	3	2.90
$(U3,M2)$	$0.2 \cdot 0.1 + 0.3 \cdot 0.4 = 0.14$	1	0.86
$(U3,M3)$	$0.1 \cdot 0.1 + 0.5 \cdot 0.4 = 0.21$	4	3.79

All errors are positive: the small initial factors predict far too low.

Step 2: user gradients. Each user sums over the movies they rated; the penalty term is $2\lambda_1 p_x = 0.2\,p_x$.

$$\begin{aligned}
\nabla p_{U1} &= -2\big[3.87(0.4, 0.1) + 1.92(0.1, 0.5)\big] + 0.2(0.3, 0.1) \\
&= -2(1.740,\ 1.347) + (0.06,\ 0.02) = (-3.420,\ -2.674), \\
\nabla p_{U2} &= -2\big[2.90(0.4, 0.1) + 2.90(0.2, 0.3)\big] + 0.2(0.2, 0.2) \\
&= -2(1.740,\ 1.160) + (0.04,\ 0.04) = (-3.440,\ -2.280), \\
\nabla p_{U3} &= -2\big[0.86(0.2, 0.3) + 3.79(0.1, 0.5)\big] + 0.2(0.1, 0.4) \\
&= -2(0.551,\ 2.153) + (0.02,\ 0.08) = (-1.082,\ -4.226).
\end{aligned}$$

Step 3: movie gradients. Each movie sums over the users who rated it, using the *old* P_0; the penalty term is $0.2\,q_i$.

$$\begin{aligned}
\nabla q_{M1} &= -2\big[3.87(0.3, 0.1) + 2.90(0.2, 0.2)\big] + 0.2(0.4, 0.1) \\
&= -2(1.741,\ 0.967) + (0.08,\ 0.02) = (-3.402,\ -1.914), \\
\nabla q_{M2} &= -2\big[2.90(0.2, 0.2) + 0.86(0.1, 0.4)\big] + 0.2(0.2, 0.3) \\
&= -2(0.666,\ 0.924) + (0.04,\ 0.06) = (-1.292,\ -1.788), \\
\nabla q_{M3} &= -2\big[1.92(0.3, 0.1) + 3.79(0.1, 0.4)\big] + 0.2(0.1, 0.5) \\
&= -2(0.955,\ 1.708) + (0.02,\ 0.10) = (-1.890,\ -3.316).
\end{aligned}$$

Step 4: update with $\eta = 0.01$ (subtracting $0.01\times$ a negative gradient increases every entry):

$$P_1 = \begin{bmatrix} 0.33420 & 0.12674 \\ 0.23440 & 0.22280 \\ 0.11082 & 0.44226 \end{bmatrix} \begin{matrix} \leftarrow U1 \\ \leftarrow U2 \\ \leftarrow U3 \end{matrix} \qquad Q_1 = \begin{bmatrix} 0.43402 & 0.11914 \\ 0.21292 & 0.31788 \\ 0.11890 & 0.53316 \end{bmatrix} \begin{matrix} \leftarrow M1 \\ \leftarrow M2 \\ \leftarrow M3 \end{matrix}$$

For example, $p_{U1} = (0.3,\ 0.1) - 0.01\,(-3.420,\ -2.674) = (0.33420,\ 0.12674)$.

Check that the step helped. The squared error on the six ratings falls from 50.587 to 49.633, and the objective (error plus penalty) from 50.678 to 49.741; the training RMSE falls from 2.904 to 2.876. A single small step with $\eta = 0.01$ makes only slow progress, and many iterations are needed.

Answer. Errors $(3.87, 1.92, 2.90, 2.90, 0.86, 3.79)$; gradients as above; P_1 and Q_1 as displayed. Every factor entry increases because every prediction was too low.

(c) Rewriting the update. With $\nabla p_{U1} = -2\sum_i e_{U1,i}\,q_i + 2\lambda_1 p_{U1}$,

$$p_{U1} \leftarrow p_{U1} - \eta\Big(-2\sum_i e_{U1,i}\,q_i + 2\lambda_1 p_{U1}\Big) = (1 - 2\eta\lambda_1)\,p_{U1} + 2\eta\sum_i e_{U1,i}\,q_i.$$

The first term *shrinks* the old vector by a fixed factor (towards the origin); the second moves it in the direction that reduces the errors.

Numbers. The data term is the same for both values of λ_1: $2\eta\sum_i e_{U1,i}q_i = 0.02\,(1.740,\ 1.347) = (0.03480,\ 0.02694)$.

λ_1	shrink factor $1 - 2\eta\lambda_1$	shrunk p_{U1}	new p_{U1}
0.1	0.998	(0.29940, 0.09980)	(0.33420, 0.12674)
0.5	0.990	(0.29700, 0.09900)	(0.33180, 0.12594)

The larger λ_1 gives a smaller p_{U1}, by exactly $2\eta(0.5-0.1)\,p_{U1} = 0.008\,(0.3,\ 0.1) = (0.0024,\ 0.0008)$. The same happens to every row of P: $p_{U2} = (0.2328,\ 0.2212)$ and $p_{U3} = (0.11002,\ 0.43906)$ instead of the values in P_1.

After one iteration and after many. After one step the effect is tiny (0.8% of p_{U1}), because η is small and the data term dominates. But the shrinkage acts in *every* iteration and compounds. The penalty alone would multiply p_{U1} by $0.998^{100} \approx 0.82$ over 100 iterations with $\lambda_1 = 0.1$, but by $0.990^{100} \approx 0.37$ with $\lambda_1 = 0.5$. At convergence the factors settle where the pull of the data balances the pull towards the origin. A larger λ_1 means shorter user vectors, more conservative predictions, and less overfitting. Users with few ratings, whose data term is weak, are affected most ("Effect of Regularization" slide).

Does λ_1 change Q_1? No, not in this iteration. ∇Q is computed from the old P_0 and involves only λ_2, so Q_1 is identical for both values of λ_1. From the second iteration on, Q is affected indirectly, because the next ∇Q uses the (smaller) updated P.

Answer. With $\lambda_1 = 0.5$: $p_{U1} = (0.33180,\ 0.12594)$ instead of $(0.33420,\ 0.12674)$. A larger λ_1 shrinks all user vectors towards 0 by the factor $1 - 2\eta\lambda_1$ per iteration; the effect is small per step but compounds over many iterations. Q_1 is unchanged in this iteration.

(d) One SGD step on $r_{U1,M1} = 4$. Only $p_{U1} = (0.3, 0.1)$ and $q_{M1} = (0.4, 0.1)$ are involved.

$$
\begin{aligned}
\xi &= 2\,(4 - 0.13) = 7.74,\\
q_{M1} &\leftarrow (0.4,\ 0.1) + 0.01\big[7.74\,(0.3,\ 0.1) - 0.1\,(0.4,\ 0.1)\big]\\
&= (0.4,\ 0.1) + 0.01\,(2.282,\ 0.764) = (0.42282,\ 0.10764),\\
p_{U1} &\leftarrow (0.3,\ 0.1) + 0.01\big[7.74\,(0.4,\ 0.1) - 0.1\,(0.3,\ 0.1)\big]\\
&= (0.3,\ 0.1) + 0.01\,(3.066,\ 0.764) = (0.33066,\ 0.10764).
\end{aligned}
$$

Both updates use the old vectors. The other 8 parameters are untouched, and the prediction $\hat{r}_{U1,M1}$ rises from 0.13 to $0.33066 \cdot 0.42282 + 0.10764 \cdot 0.10764 = 0.1514$. (In this rule the factor 2 of the penalty's derivative is absorbed into λ_1, λ_2, as the lecture notes, so the penalty is applied with weight λ rather than 2λ. It is also applied once per rating, so a user or movie with many ratings is shrunk many times per pass. SGD with $\lambda = 0.1$ therefore does not minimize exactly the objective of part (b), and the numbers of (b) and (d) should not be compared directly.)

Answer. $q_{M1} = (0.42282,\ 0.10764)$ and $p_{U1} = (0.33066,\ 0.10764)$; all other rows keep their P_0, Q_0 values.

Cost. Input size: $|R|$ known ratings, $|X|$ users, $|S|$ movies and k factors; the elementary operations are arithmetic operations on numbers.

- *One GD iteration* processes **all** $|R|$ ratings (here 6). For each rating it computes the dot product $q_i \cdot p_x$ (k multiplications and $k-1$ additions) and the error, and adds $e_{xi}q_i$ to user x's gradient and $e_{xi}p_x$ to movie i's gradient ($2k$ multiplications and $2k$ additions). That is about $6k$ operations per rating. Adding the penalty and updating all $(|X| + |S|)\,k$ parameters costs a few operations each, so an iteration costs about $6k|R| + 4k(|X| + |S|) = \Theta(k\,|R|)$, since each user and movie has at least one rating.
- *One SGD step* processes **one** rating and changes $2k$ parameters: a dot product (about $2k$ operations), ξ, and two vector updates of about $5k$ operations each, so about $12k$ operations, i.e. $\Theta(k)$, independent of $|R|$.

Answer. GD: all $|R|$ ratings and $\Theta(k|R|)$ operations per iteration; SGD: one rating and $\Theta(k)$ operations per step. On Netflix ($|R| \approx 10^8$, e.g. $k = 100$) one GD iteration costs on the order of 6×10^{10} operations before any parameter moves, whereas in the same time SGD makes about 10^8 (noisier) updates, one per rating.

This is why the lecture calls gradient computation slow and prefers SGD: it needs more steps, but each step is far cheaper, so it converges faster in wall-clock time.

Exercise 8: Predicting Ratings with LFMs

Ex8: Predicting Ratings with LFMs

(a) Using the matrices P_1 and Q_1 from Ex7(b) (with $\lambda_1 = \lambda_2 = 0.1$), predict the missing ratings for $M2$ for user $U1$ and for $M3$ for user $U2$. Compare them with the known ratings: what do they tell you about the model after one iteration?

(b) The true values of these two ratings are unknown. Discuss how you would evaluate the accuracy of the model's predictions.

(c) The lecture improves the model with biases: $\hat{r}_{xi} = \mu + b_x + b_i + q_i \cdot p_x$. Estimate μ, b_x and b_i from the six known ratings as in "CF: Common Practice" (μ: overall mean rating; b_x = (mean rating of user x) $-\ \mu$; b_i = (mean rating of movie i) $-\ \mu$), and compute the baseline part $\mu + b_x + b_i$ for the same two missing ratings. Why does the biased model start from much better predictions than (a), and what is left for the term $q_i \cdot p_x$ to learn?

Solution

(a) From Ex7(b): $p_{U1} = (0.33420, 0.12674)$, $p_{U2} = (0.23440, 0.22280)$, $q_{M2} = (0.21292, 0.31788)$, $q_{M3} = (0.11890, 0.53316)$.

$$\hat{r}_{U1,M2} = q_{M2} \cdot p_{U1} = 0.21292 \cdot 0.33420 + 0.31788 \cdot 0.12674 = 0.07116 + 0.04029 \approx 0.111,$$
$$\hat{r}_{U2,M3} = q_{M3} \cdot p_{U2} = 0.11890 \cdot 0.23440 + 0.53316 \cdot 0.22280 = 0.02787 + 0.11879 \approx 0.147.$$

Answer. $\hat{r}_{U1,M2} \approx 0.11$ and $\hat{r}_{U2,M3} \approx 0.15$ stars.

Interpretation. These are not usable predictions. The known ratings range from 1 to 4 (mean 2.83), yet both predictions are below even the lowest possible rating of 1 star. They have barely moved from their initial values (0.09 and 0.12), and the training RMSE is still 2.88. After one iteration the model mainly reflects its small random initialization: it is far from the minimum of the objective. Gradient descent must be run for many iterations, until the objective (or the error on a validation set) stops decreasing, before the predictions mean anything.

(b) We cannot compare a prediction with a value we do not know, so we evaluate the *model* on ratings we do know but hide from it:

1. **Hold out** part of the known ratings as a test set before training (as Netflix did, preferably the *most recent* ratings, which mimics predicting the future).
2. **Train** P and Q on the remaining ratings only.
3. **Predict** the held-out ratings and compare them with their true values, for example by

   $$RMSE = \sqrt{\frac{1}{|R|} \sum_{(x,i) \in R} (\hat{r}_{xi} - r_{xi})^2}$$

   over the held-out set R. If the system shows a top-k list, also use ranking metrics such as precision@k (Exercise 4).
4. **Tune** k, λ_1, λ_2, η and the number of iterations on a separate validation set (or by cross-validation), never on the test set.

A low held-out error gives confidence that the predictions for the truly unknown cells, such as $\hat{r}_{U1,M2}$ and $\hat{r}_{U2,M3}$, are accurate too. In this tiny example, with only six ratings, holding out even one leaves little to train on. Leave-one-out or k-fold cross-validation would be used, and the estimate would still be very noisy.

(c) Means. $\mu = (4+2+3+3+1+4)/6 = 17/6 \approx 2.833$.

user	mean	b_x	movie	mean	b_i
U1	$(4+2)/2 = 3$	$+1/6 \approx +0.167$	M1	$(4+3)/2 = 3.5$	$+2/3 \approx +0.667$
U2	$(3+3)/2 = 3$	$+1/6 \approx +0.167$	M2	$(3+1)/2 = 2$	$-5/6 \approx -0.833$
U3	$(1+4)/2 = 2.5$	$-1/3 \approx -0.333$	M3	$(2+4)/2 = 3$	$+1/6 \approx +0.167$

Baselines of the two missing cells.

$$b_{U1,M2} = \tfrac{17}{6} + \tfrac{1}{6} - \tfrac{5}{6} = \tfrac{13}{6} \approx 2.17, \qquad b_{U2,M3} = \tfrac{17}{6} + \tfrac{1}{6} + \tfrac{1}{6} = \tfrac{19}{6} \approx 3.17.$$

Answer. $\mu = 17/6$; $b_{U1} = b_{U2} = 1/6$, $b_{U3} = -1/3$; $b_{M1} = 2/3$, $b_{M2} = -5/6$, $b_{M3} = 1/6$. Baselines: $\mu + b_{U1} + b_{M2} = 13/6 \approx 2.17$ and $\mu + b_{U2} + b_{M3} = 19/6 \approx 3.17$.

Why the biased model starts much better. With small initial factors, $q_i \cdot p_x \approx 0$, so the biased model starts *at the baseline*, which is already on the rating scale. U1 rates a bit above average, but M2 is a poorly rated movie, hence 2.17; U2 is slightly generous and M3 is slightly above average, hence 3.17. The plain model starts at 0 stars instead. On the six known ratings the baseline alone has RMSE $\sqrt{29/36} \approx 0.90$, against 2.88 for the plain model after one gradient step.

What is left for $q_i \cdot p_x$. Only the user-movie *interaction*: the residuals $r_{xi} - (\mu + b_x + b_i)$, which on the known ratings are

$$(U1, M1)\colon +\tfrac{1}{3}, \quad (U1, M3)\colon -\tfrac{7}{6}, \quad (U2, M1)\colon -\tfrac{2}{3}, \quad (U2, M2)\colon +\tfrac{5}{6}, \quad (U3, M2)\colon -\tfrac{2}{3}, \quad (U3, M3)\colon +\tfrac{4}{3}.$$

For example, U3 likes M3 more than the two biases explain, and U1 likes it less. These residuals are small and centered at 0, which is exactly what small initial factors and regularization towards 0 (i.e. towards the baseline) assume. In the full model the biases are learned together with the factors, penalized by λ_3, λ_4, and the factors are trained on these residuals. Simply adding the factors of Ex7, which were fitted to the raw ratings, to the baseline would not be consistent. The lecture's performance plot shows that adding the biases buys more than tripling the number of parameters.